

第 2 章

图像处理基础

本章主要介绍图像的基本表示方法、像素的访问和操作、感兴趣区域处理、通道处理等知识点。需要强调的是，使用面向 Python 的 OpenCV（OpenCV for Python）必须熟练掌握 Numpy 库，尤其是 Numpy.array 库，Numpy.array 库是 Python 处理图像的基础。

2.1 图像的基本表示方法

本节主要讨论二值图像、灰度图像、彩色图像的基本表示方法。

1. 二值图像

二值图像是指仅仅包含黑色和白色两种颜色的图像。

在计算机中，通过一个栅格状排列的数据集（矩阵）来表示和处理图像。例如，图 2-1 是一个字母 A 的图像，计算机在处理该图像时，会首先将其划分为一个个的小方块，每一个小方块就是一个独立的处理单位，称为像素点。接下来，计算机将会将其中的白色像素点（白色小方块区域）处理为“1”，将黑色像素点（黑色小方块区域）处理为“0”，以方便进行后续的存储和处理等操作。

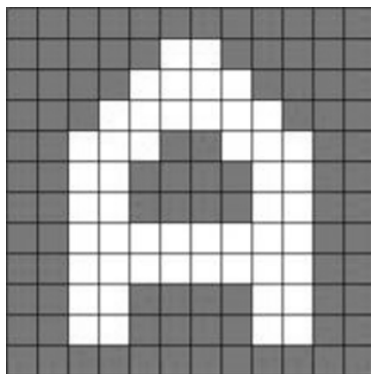


图 2-1 字母 A 的图像

按照上述处理方式，图 2-1 中的字母 A 在计算机内的存储形式如图 2-2 所示。



0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	1	1	0	0	0	0	0	0
0	0	0	0	1	1	1	1	0	0	0	0	0
0	0	0	1	1	1	1	1	1	0	0	0	0
0	0	1	1	1	0	0	1	1	1	0	0	0
0	0	1	1	0	0	0	0	1	1	0	0	0
0	0	1	1	0	0	0	0	1	1	0	0	0
0	0	1	1	1	1	1	1	1	1	0	0	0
0	0	1	1	1	1	1	1	1	1	0	0	0
0	0	1	1	0	0	0	0	1	1	0	0	0
0	0	1	1	0	0	0	0	1	1	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0

图 2-2 计算机内字母 A 的存储形式

上述图像比较简单，图像内只有黑色和白色两种不同的颜色，因此只使用一个比特位（0 或者 1）就能表示。

2. 灰度图像

二值图像表示起来简单方便，但是因为其仅有黑白两种颜色，所表示的图像不够细腻。如果想要表现更多的细节，就需要使用更多的颜色。例如，图 2-3 中的 lena 图像是一幅灰度图像，它采用了更多的数值以体现不同的颜色，因此该图像的细节信息更丰富。



图 2-3 lena 图像

通常，计算机会将灰度处理为 256 个灰度级，用数值区间[0, 255]来表示。其中，数值“255”表示纯白色，数值“0”表示纯黑色，其余的数值表示从纯白到纯黑之间不同级别的灰度。

用于表示 256 个灰度级的数值 0~255，正好可以用一个字节（8 位二进制值）来表示。表 2-1 所示的是部分二进制值所对应的十进制值及灰度颜色。

表 2-1 部分灰度级及所对应的值

二进制值	十进制值	颜色	备注
0000 0000	0		纯黑色
0001 0000	16		深灰色
0010 0000	32		深灰色
0100 0000	64		深灰色
1000 0000	128		浅灰色
1010 0110	166		浅灰色

续表

二进制值	十进制值	颜色	备注
1011 1100	188		浅灰色
1100 1000	200		浅灰色
1110 0001	225		浅灰色
1111 1111	255		纯白色

按照上述方法,图 2-3 中的图像需要使用一个各行各列的数值都在 $[0, 255]$ 之间的矩阵来表示。例如,图 2-4 就是图 2-3 的 lena 图像中部分区域的数值表示形式。

134	224	185	46	152	105	235	95	56	51
192	206	137	52	58	254	246	101	42	182
3	247	38	246	0	212	138	85	53	217
74	11	134	41	185	169	5	26	121	86
202	2	219	22	106	225	121	139	123	198
232	235	131	165	77	189	23	56	42	16
133	37	67	125	58	232	251	125	243	149
165	149	170	139	8	188	21	130	58	223
237	14	171	91	131	156	28	87	126	106
10	142	250	150	10	73	159	117	238	196
85	2	196	247	72	2	252	121	179	202
200	84	255	170	212	13	88	82	125	108
15	184	4	9	205	180	3	242	89	51

图 2-4 部分 lena 图像的数值表示

有些情况下,也会使用 8 位二进制值来表示一幅二值图像。这种情况下,使用灰度值 255 表示白色、灰度值 0 表示黑色。此时,该二值图像内仅有数值 0 和数值 255 两种类型的灰度值(灰度级),不存在其他灰度值的像素点。

3. 彩色图像

相比二值图像和灰度图像,彩色图像是更常见的一类图像,它能表现更丰富的细节信息。

神经生理学实验发现,在视网膜上存在三种不同的颜色感受器,能够感受三种不同的颜色:红色、绿色和蓝色,即三基色。自然界中常见的各种色光都可以通过将三基色按照一定的比例混合构成。除此以外,从光学角度出发,可以将颜色解析为主波长、纯度、明度等。从心理学和视觉角度出发,可以将颜色解析为色调、饱和度、亮度等。通常,我们将上述采用不同的方式表述颜色的模式称为色彩空间,或者颜色空间、颜色模式等。

虽然不同的色彩空间具有不同的表示方式,但是各种色彩空间之间可以根据需要按照公式进行转换。这里仅仅介绍较为常用的 RGB 色彩空间。

在 RGB 色彩空间中,存在 R(red, 红色)通道、G(green, 绿色)通道和 B(blue, 蓝色)通道,共三个通道。每个色彩通道值的范围都在 $[0, 255]$ 之间,我们用这三个色彩通道的组合表示颜色。

以比较通俗的方式来解释就是,有三个油漆桶,分别装了红色、绿色、蓝色的油漆,我们



分别从每个油漆桶中取容量为 0~255 个单位的不等量的油漆，将三种油漆混合就可以调配出一种新的颜色。三种油漆经过不同的组合，共可以调配出所有常见的 $256 \times 256 \times 256 = 16\,777\,216$ 种颜色。

表 2-2 展示了不同的 RGB 值所对应的颜色。

表 2-2 RGB 值及颜色示例

R 值	G 值	B 值	RGB 值	颜色
0	0	0	(0,0,0)	纯黑色
255	255	255	(255,255,255)	纯白色
255	0	0	(255,0,0)	红色
0	255	0	(0,255,0)	绿色
0	0	255	(0,0,255)	蓝色
114	141	216	(114,141,216)	天蓝色
139	69	19	(139,69,19)	棕色

例如，对于图 2-5 左侧的彩色图像，可以理解为由右侧的 R 通道、G 通道、B 通道三个通道构成。其中，每一个通道都可以理解为一个独立的灰度图像。左侧彩色图像中的白色方块内的区域对应右侧三个通道的三个矩阵，白色方块左上角顶点的 RGB 值为(205,89,68)。

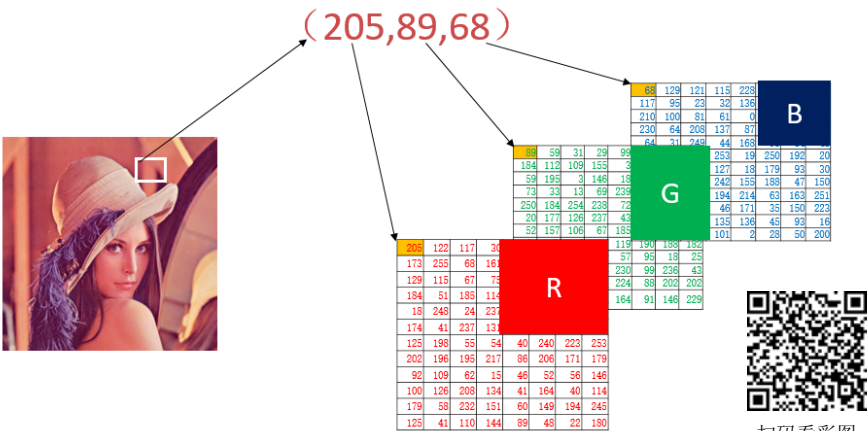


图 2-5 图像数据展示

因此，通常用一个三维数组来表示一幅 RGB 色彩空间的彩色图像。

一般情况下，在 RGB 色彩空间中，图像通道的顺序是 R→G→B，即第 1 个通道是 R 通道，第 2 个通道是 G 通道，第 3 个通道是 B 通道。需要特别注意的是，在 OpenCV 中，通道的顺序是 B→G→R，即：

- 第 1 个通道保存 B 通道的信息。
- 第 2 个通道保存 G 通道的信息。
- 第 3 个通道保存 R 通道的信息。

在图像处理过程中，可以根据需要对图像的通道顺序进行转换。除此以外，还可以根据需要对不同色彩空间的图像进行类型转换，例如，将灰度图像处理为二值图像，将彩色图像处理为灰度图像等。

2.2 像素处理

像素是图像构成的基本单位，像素处理是图像处理的基本操作，可以通过位置索引的形式对图像内的元素进行访问、处理。

1. 二值图像及灰度图像

需要说明的是，在 OpenCV 中，最小的数据类型是无符号的 8 位数。因此，在 OpenCV 中实际上并没有二值图像这种数据类型，二值图像经常是通过处理得到的，然后使用 0 表示黑色，使用 255 表示白色。

可以将二值图像理解为特殊的灰度图像，这里仅以灰度图像为例讨论像素的读取和修改。通过 2.1 节的分析可知，可以将图像理解为一个矩阵，在面向 Python 的 OpenCV（OpenCV for Python）中，图像就是 Numpy 库中的数组。一个 OpenCV 灰度图像是一个二维数组，可以使用表达式访问其中的像素值。例如，可以使用 `image[0,0]` 访问图像 `image` 第 0 行第 0 列位置上的像素点。第 0 行第 0 列位于图像的左上角，其中第 1 个索引表示第 0 行，第 2 个索引表示第 0 列。

为了方便理解，我们首先使用 Numpy 库来生成一个 8×8 大小的数组，用来模拟一个黑色图像，并对其进行简单处理。

【例 2.1】 使用 Numpy 库生成一个元素值都是 0 的二维数组，用来模拟一幅黑色图像，并对其进行访问、修改。

分析：使用 Numpy 库中的函数 `zeros()` 可以生成一个元素值都是 0 的数组，并可以直接使用数组的索引对其进行访问、修改。

根据题目要求及分析，编写代码如下：

```
import cv2
import numpy as np
img=np.zeros((8,8),dtype=np.uint8)
print("img=\n",img)
cv2.imshow("one",img)
print("读取像素点 img[0,3]=",img[0,3])
img[0,3]=255
print("修改后 img=\n",img)
print("读取修改后像素点 img[0,3]=",img[0,3])
cv2.imshow("two",img)
cv2.waitKey()
cv2.destroyAllWindows()
```

代码分析如下。

- 使用函数 `zeros()` 生成了一个 8×8 大小的二维数组，其中所有的值都是 0，数值类型是 `np.uint8`。根据该数组的属性，可以将其看成一个黑色的图像。
- 语句 `img[0,3]` 访问的是 `img` 第 0 行第 3 列的像素点，需要注意的是，行序号、列序号都是从 0 开始的。



- 语句 `img[0,3]=255` 将 `img` 中第 0 行第 3 列的像素点的像素值设置为“255”。

运行上述程序，会出现名为 `one` 和 `two` 的两个非常小的窗口，其中：

- 名为 `one` 的窗口是一个纯黑色的图像。
- 名为 `two` 的窗口在顶部靠近中间位置有一个白点（对应修改后的值 255），其他地方也都是纯黑的图像。

同时，在控制台会输出如下内容：

```
img=
[[0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]
 [0 0 0 0 0 0 0 0]]
读取像素点 img[0,3]= 0
修改后 img=
[[ 0  0  0 255  0  0  0  0]
 [ 0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0  0]]
读取修改后像素点 img[0,3]= 255
```

通过本例中两个窗口显示的图像可知，二维数组与图像之间存在对应关系。因为窗口太小，在纸质图书上可能无法直接观察效果，请大家在计算机上运行上述程序，观察效果。

【例 2.2】 读取一个灰度图像，并对其像素进行访问、修改。

根据题目要求，编写代码如下：

```
import cv2
img=cv2.imread("lena.bmp",0)
cv2.imshow("before",img)
for i in range(10,100):
    for j in range(80,100):
        img[i,j]=255
cv2.imshow("after",img)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，使用了一个嵌套循环语句，将图像 `img` 中“第 10 行到 99 行”与“第 80 列到 99 列”交叉区域内的像素值设置为 255。从图像 `img` 上来看，该交叉区域被设置为白色。

运行程序，结果如图 2-6 所示，其中：

- 左图是读取的原始图像。
- 右图是经过修改后的图像。

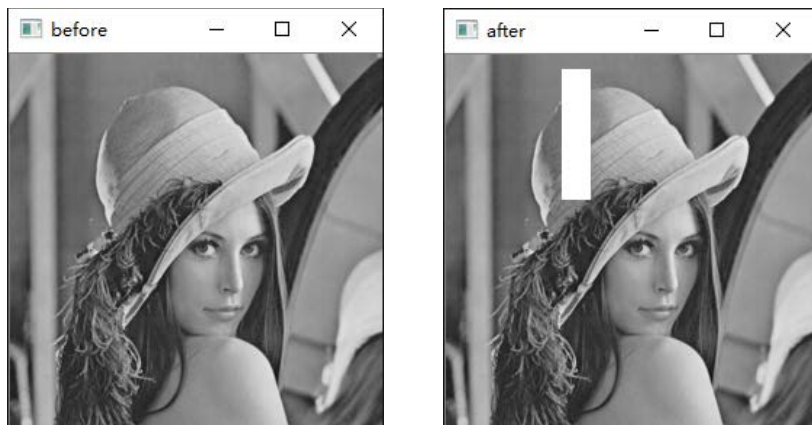


图 2-6 像素修改示例

2. 彩色图像

RGB 模式的彩色图像在读入 OpenCV 内进行处理时，会按照行方向依次读取该 RGB 图像的 B 通道、G 通道、R 通道的像素点，并将像素点以行为单位存储在 ndarray 的列中。例如，有一幅大小为 R 行×C 列的原始 RGB 图像，其在 OpenCV 内以 BGR 模式的三维数组形式存储，如图 2-7 所示。

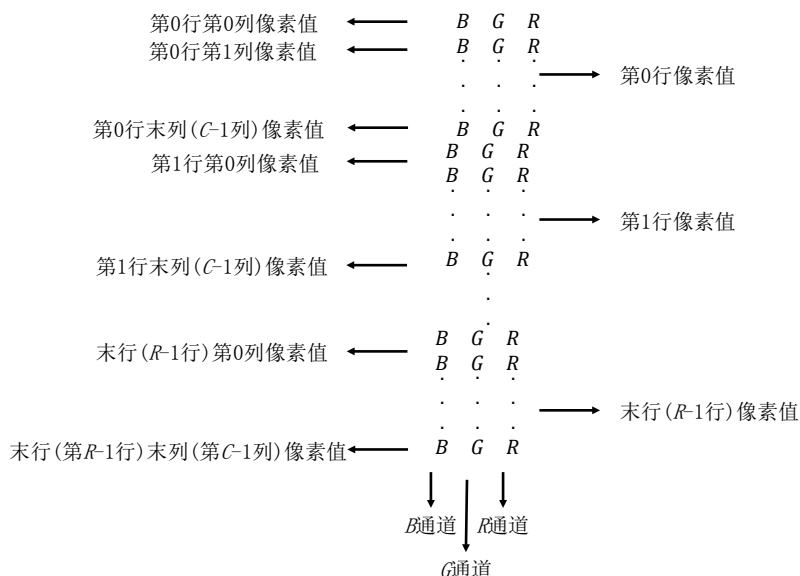


图 2-7 RGB 图像以三维数组形式存储的情况说明

可以使用表达式访问数组内的值。例如，可以使用 `image[0,0,0]` 访问图像 `image` 的 B 通道内的第 0 行第 0 列上的像素点，式中：



- 第 1 个索引表示第 0 行。
- 第 2 个索引表示第 0 列。
- 第 3 个索引表示第 0 个颜色通道。

根据上述分析可知，假设有一个红色（其 R 通道值为 255，G 通道值为 0，B 通道值为 0）图像，不同的访问方式得到的值如下。

- `img[0,0]`: 访问图像 `img` 第 0 行第 0 列像素点的 BGR 值。图像是 BGR 格式的，得到的数值为 `[0,0,255]`。
- `img[0,0,0]`: 访问图像 `img` 第 0 行第 0 列第 0 个通道的像素值。图像是 BGR 格式的，所以第 0 个通道是 B 通道，会得到 B 通道内第 0 行第 0 列的位置所对应的值 0。
- `img[0,0,1]`: 访问图像 `img` 第 0 行第 0 列第 1 个通道的像素值。图像是 BGR 格式的，所以第 1 个通道是 G 通道，会得到 G 通道内第 0 行第 0 列的位置所对应的值 0。
- `img[0,0,2]`: 访问图像 `img` 第 0 行第 0 列第 2 个通道的像素值。图像是 BGR 格式的，所以第 2 个通道是 R 通道，会得到 R 通道内第 0 行第 0 列的位置所对应的值 255。

为了方便理解，我们首先使用 Numpy 库来生成一个 $2 \times 4 \times 3$ 大小的数组，用它模拟一幅黑色图像，并对其进行简单处理。

【例 2.3】 使用 Numpy 生成三维数组，用来观察三个通道值的变化情况。

根据题目要求，编写代码如下：

```
import numpy as np
import cv2
#-----蓝色通道值-----
blue=np.zeros((300,300,3),dtype=np.uint8)
blue[:, :, 0]=255
print("blue=\n",blue)
cv2.imshow("blue",blue)
#-----绿色通道值-----
green=np.zeros((300,300,3),dtype=np.uint8)
green[:, :, 1]=255
print("green=\n",green)
cv2.imshow("green",green)
#-----红色通道值-----
red=np.zeros((300,300,3),dtype=np.uint8)
red[:, :, 2]=255
print("red=\n",red)
cv2.imshow("red",red)
#-----释放窗口-----
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，分别生成了 `blue`、`green`、`red` 三个数组，其初始值都是 0。接下来，分别改变各通道值。

- 针对数组 `blue`，将其第 0 个通道的值设置为 255。从图像角度来看，图像 `blue` 的 B 通道

值为 255，其余两个通道值为 0，因此图像 blue 为蓝色图像。

- 针对数组 green，将其第 1 个通道的值设置为 255。从图像角度来看，图像 green 的 G 通道值为 255，其余两个通道值为 0，因此图像 green 为绿色图像。
- 针对数组 red，将其第 2 个通道的值设置为 255。从图像角度来看，图像 red 的 R 通道值为 255，其余两个通道值为 0，因此图像 red 为红色图像。

运行上述程序，会显示颜色为蓝色、绿色、红色的三幅图像，分别对应数组 blue、数组 green、数组 red。因为黑白印刷无法显示彩色图像，所以请读者运行程序后观察结果。

除了显示图像，还会显示每个数组的输出值，部分输出结果如图 2-8 所示。

<pre>blue= [[[255 0 0] [255 0 0] [255 0 0] ... [255 0 0] [255 0 0] [255 0 0] ... [255 0 0] [255 0 0] [255 0 0]]]</pre>	<pre>green= [[[0 255 0] [0 255 0] [0 255 0] ... [0 255 0] [0 255 0] [0 255 0] ... [0 255 0] [0 255 0] [0 255 0]]]</pre>	<pre>red= [[[0 0 255] [0 0 255] [0 0 255] ... [0 0 255] [0 0 255] [0 0 255] ... [0 0 255] [0 0 255] [0 0 255]]]</pre>
--	---	---

图 2-8 【例 2.3】程序的部分输出结果

【例 2.4】使用 Numpy 生成一个三维数组，用来观察三个通道值的变化情况。

根据题目要求，编写代码如下：

```
import numpy as np
import cv2
img=np.zeros((300,300,3),dtype=np.uint8)
img[:,0:100,0]=255
img[:,100:200,1]=255
img[:,200:300,2]=255
print("img=\n",img)
cv2.imshow("img",img)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，会显示如图 2-9 所示的图像。由于本书为黑白印刷，所以为了更好地观察运行效果，请大家亲自上机运行程序。

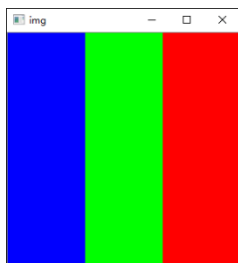


图 2-9 【例 2.4】程序的运行结果



除了显示图像，还会显示 `img` 值的情况，部分输出结果如图 2-10 所示。

```
img=
[[[255  0  0]
  [255  0  0]
  [255  0  0]
  ...
  [ 0  0 255]
  [ 0  0 255]
  [ 0  0 255]]

 [[255  0  0]
  [255  0  0]
  [255  0  0]
  ...
  [ 0  0 255]
  [ 0  0 255]
  [ 0  0 255]]
```

图 2-10 【例 2.4】程序的部分输出结果

【例 2.5】使用 Numpy 生成一个三维数组，用来模拟一幅 BGR 模式的彩色图像，并对其进行访问、修改。

分析：使用 Numpy 中的 `zeros()`函数可以生成一个元素值都是 0 的数组。可以直接使用数组的索引形式对其进行访问、修改。

根据题目要求及分析，编写代码如下：

```
1. import numpy as np
2. img=np.zeros((2,4,3),dtype=np.uint8)
3. print("img=\n",img)
4. print("读取像素点 img[0,3]=",img[0,3])
5. print("读取像素点 img[1,2,2]=",img[1,2,2])
6. img[0,3]=255
7. img[0,0]=[66,77,88]
8. img[1,1,1]=3
9. img[1,2,2]=4
10. img[0,2,0]=5
11. print("修改后 img\n",img)
12. print("读取修改后像素点 img[1,2,2]=",img[1,2,2])
```

本程序进行了如下操作。

- 第 2 行使用 `zeros()`生成一个 $2 \times 4 \times 3$ 大小的数组，其对应一个“2 行 4 列 3 个通道”的 BGR 图像。
- 第 3 行使用 `print` 语句显示（打印）当前图像（数组）的值。
- 第 4 行中的 `img[0,3]`语句会访问第 0 行第 3 列位置上的 B 通道、G 通道、R 通道三个像素点。
- 第 5 行的 `img[1,2,2]`语句会访问第 1 行第 2 列第 2 个通道位置上的像素点。
- 第 6 行的 `img[0,3]=255` 语句会修改 `img` 中第 0 行第 3 列位置上的像素值，该位置上的 B 通道、G 通道、R 通道三个像素点的值都会被修改为 255。

- 第7行的 `img[0,0]=[66,77,88]` 语句会修改 `img` 中第0行第0列位置上的B通道、G通道、R通道上三个像素点的值，将它们修改为[66,77,88]。
- 第8行的 `img[1,1,1]=3` 语句会修改 `img` 中第1行第1列第1个通道（G通道）位置上的像素值，将其修改为3。
- 第9行的 `img[1,2,2]=4` 语句会修改 `img` 中第1行第2列第2个通道（R通道）位置上的像素值，将其修改为4。
- 第10行的 `img[0,2,0]=5` 语句会修改 `img` 中第0行第2列第0个通道（B通道）位置上的像素值，将其修改为5。
- 最后两行使用 `print` 语句观察 `img` 和 `img[1,2,2]` 的值。

运行上述程序，会在控制台输出如下结果：

```
img=
[[[0 0 0]
  [0 0 0]
  [0 0 0]
  [0 0 0]]
 [[0 0 0]
  [0 0 0]
  [0 0 0]
  [0 0 0]]]
读取像素点 img[0,3]= [0 0 0]
读取像素点 img[1,2,2]= 0
修改后 img
[[[ 66  77  88]
  [ 0  0  0]
  [ 5  0  0]
  [255 255 255]]
 [[ 0  0  0]
  [ 0  3  0]
  [ 0  0  4]
  [ 0  0  0]]]
读取修改后像素点 img[1,2,2]= 4
```

在本例中，为了方便说明问题，设置的数组比较小。在实际中可以定义稍大的数组，并使用 `cv2.imshow()` 将其显示出来，进一步观察处理结果，加深理解。

【例 2.6】 读取一幅彩色图像，并对其像素进行访问、修改。

根据题目要求，编写代码如下：

```
1. import cv2
2. img=cv2.imread("lenacolor.png")
3. cv2.imshow("before",img)
4. print("访问 img[0,0]=",img[0,0])
5. print("访问 img[0,0,0]=",img[0,0,0])
6. print("访问 img[0,0,1]=",img[0,0,1])
7. print("访问 img[0,0,2]=",img[0,0,2])
```



```

8. print("访问 img[50,0]=",img[50,0])
9. print("访问 img[100,0]=",img[100,0])
10. #区域 1
11. for i in range(0,50):
12.     for j in range(0,100):
13.         for k in range(0,3):
14.             img[i,j,k]=255 #白色
15. #区域 2
16. for i in range(50,100):
17.     for j in range(0,100):
18.         img[i,j]=[128,128,128] #灰色
19. #区域 3
20. for i in range(100,150):
21.     for j in range(0,100):
22.         img[i,j]=0 #黑色
23. cv2.imshow("after",img)
24. print("修改后 img[0,0]=",img[0,0])
25. print("修改后 img[0,0,0]=",img[0,0,0])
26. print("修改后 img[0,0,1]=",img[0,0,1])
27. print("修改后 img[0,0,2]=",img[0,0,2])
28. print("修改后 img[50,0]=",img[50,0])
29. print("修改后 img[100,0]=",img[100,0])
30. cv2.waitKey()
31. cv2.destroyAllWindows()

```

上述程序进行了如下操作。

- 第 2 行使用 `imread()` 函数读取当前目录下的一幅彩色 RGB 图像。
- 第 4 行的 `img[0,0]` 语句会访问 `img` 中第 0 行第 0 列位置上的 B 通道、G 通道、R 通道三个像素点。
- 第 5~7 行分别会访问 `img` 中第 0 行第 0 列位置上的 B 通道、G 通道、R 通道三个像素点。
- 第 8 行的 `img[50,0]` 语句会访问第 50 行第 0 列位置上的 B 通道、G 通道、R 通道三个像素点。
- 第 9 行的 `img[100,0]` 语句会访问第 100 行第 0 列位置上的 B 通道、G 通道、R 通道三个像素点。
- 第 10~14 行使用三个 `for` 语句的嵌套循环，对图像左上角区域（即“第 0 行到第 49 行”与“第 0 列到第 99 列”的行列交叉区域，我们称之为区域 1）内的像素值进行设定。借助 `img[i, j, k]=255` 语句将该区域内的三个通道的像素值都设置为 255，让该区域变为白色。
- 第 15~18 行使用两个 `for` 语句的嵌套循环，对图像左上角位于区域 1 正下方的区域（即“第 50 行到第 99 行”与“第 0 列到第 99 列”的行列交叉区域，我们称之为区域 2）内的像素值进行设定。借助 `img[i, j]=[128,128,128]` 语句将该区域内的三个通道的像素值都设置为 128，让该区域变为灰色。

- 第 19~21 行使用两个 for 语句的嵌套循环，对图像左上角位于区域 2 正下方的区域（即“第 100 行到第 149 行”与“第 0 列到第 99 列”的行列交叉区域，我们称之为区域 3）内的像素值进行设定。借助 `img[i, j]=0` 语句将该区域内的三个通道的像素值都设置为 0，让该区域变为黑色。

运行程序，结果如图 2-11 所示，其中左图是读取的原始图像，右图是经过修改后的图像。

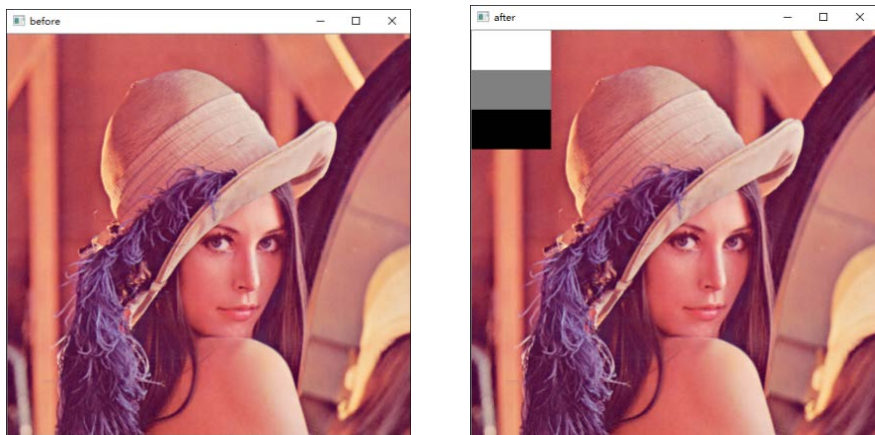


图 2-11 【例 2.6】程序的运行结果

同时，在控制台会输出如下内容：

```
访问 img[0,0]= [125 137 226]
访问 img[0,0,0]= 125
访问 img[0,0,1]= 137
访问 img[0,0,2]= 226
访问 img[50,0]= [114 136 230]
访问 img[100,0]= [ 75  55 155]
修改后 img[0,0]= [255 255 255]
修改后 img[0,0,0]= 255
修改后 img[0,0,1]= 255
修改后 img[0,0,2]= 255
修改后 img[50,0]= [128 128 128]
修改后 img[100,0]= [0 0 0]
```

2.3 使用 numpy.array 访问像素

`numpy.array` 提供了 `item()` 和 `itemset()` 函数来访问和修改像素值，而且这两个函数都是经过优化处理的，能够更大幅度地提高处理效率。在访问及修改像素点的值时，利用 `numpy.array` 提供的函数比直接使用索引要快得多，同时，这两个函数的可读性也更好。

1. 二值图像及灰度图像

可以将二值图像理解为特殊的灰度图像，所以这里仅以灰度图像为例讨论像素点值的读取和修改。



函数 `item()` 能够更加高效地访问图像的像素点，该函数的语法格式为：

`item(行,列)`

函数 `itemset()` 可以用来修改像素值，其语法格式为：

`itemset(索引值, 新值)`

为了便于理解，我们首先使用 Numpy 库生成一个 5×5 大小的随机数组，用来模拟一幅灰度图像，并对其进行简单的处理。

【例 2.7】 使用 Numpy 生成一个二维随机数组，用来模拟一幅灰度图像，并对其像素进行访问、修改。

分析：使用 Numpy 中的 `random.randint` 可以生成一个随机数组，该随机数组对应一幅灰度图像。然后分别使用函数 `item()` 及函数 `itemset()` 对其像素进行访问、修改。

根据题目要求及分析，编写代码如下：

```
import numpy as np
img=np.random.randint(10,99,size=[5,5],dtype=np.uint8)
print("img=\n",img)
print("读取像素点 img.item(3,2)=",img.item(3,2))
img.itemset((3,2),255)
print("修改后 img=\n",img)
print("修改后像素点 img.item(3,2)=",img.item(3,2))
```

运行程序，控制台输出结果如下：

```
img=
[[69 11 79 35 84]
 [87 10 63 12 20]
 [31 67 68 16 45]
 [81 26 61 51 28]
 [33 94 18 71 53]]
读取像素点 img.item(3,2)= 61
修改后 img=
[[ 69  11  79  35  84]
 [ 87  10  63  12  20]
 [ 31  67  68  16  45]
 [ 81  26 255  51  28]
 [ 33  94  18  71  53]]
修改后像素点 img.item(3,2)= 255
```

通过观察输出结果可以发现，语句 `img.itemset((3,2),255)` 将图像第 3 行第 2 列位置上的像素值修改为 255 了。

在例 2.7 中，为方便读者仔细观察数组内的每一个值，生成的数组规模（尺寸）较小。在实际使用中，可以利用随机函数生成更大尺寸的随机数组，并使用函数 `imshow()` 观察随机数组对应的灰度图像。

【例 2.8】 生成一个灰度图像，让其中的像素值均为随机数。

根据题目要求，编写代码如下：

```
import numpy as np
import cv2
img=np.random.randint(0,256,size=[256,256],dtype=np.uint8)
cv2.imshow("demo",img)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，可以生成一幅 256×256 大小的灰度图像，如图 2-12 所示。

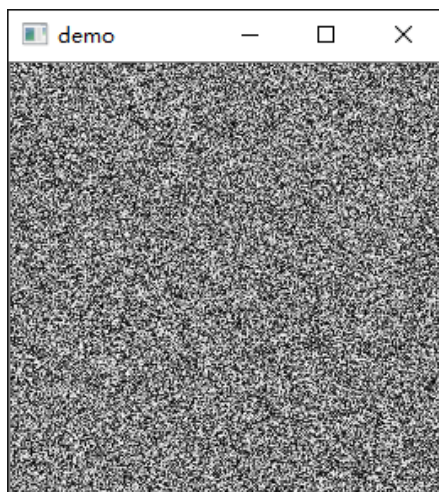


图 2-12 随机灰度图像

【例 2.9】读取一幅灰度图像，并对其像素值进行访问、修改。

根据题目要求，编写代码如下：

```
import cv2
img=cv2.imread("lena.bmp",0)
#测试读取、修改单个像素值
print("读取像素点 img.item(3,2)=",img.item(3,2))
img.itemset((3,2),255)
print("修改后像素点 img.item(3,2)=",img.item(3,2))
#测试修改一个区域的像素值
cv2.imshow("before",img)
for i in range(10,100):
    for j in range(80,100):
        img.itemset((i,j),255)
cv2.imshow("after",img)
cv2.waitKey()
cv2.destroyAllWindows()
```

本程序首先修改了一个像素点的像素值：使用 `item()` 函数读取了第 3 行第 2 列位置上的像素值；接下来使用 `itemset()` 函数对该像素值进行了修改。

接下来使用嵌套循环语句修改了一个区域的像素值，将位于“第 10 行到第 99 行”和“第 80 列到第 99 列”的行列交叉区域的像素值设置（修改）为 255，即让该区域显示为白色。

运行上述程序，显示如图 2-13 所示图像，其中，左图是读取的原始图像，右图是修改部分像素后得到的图像。

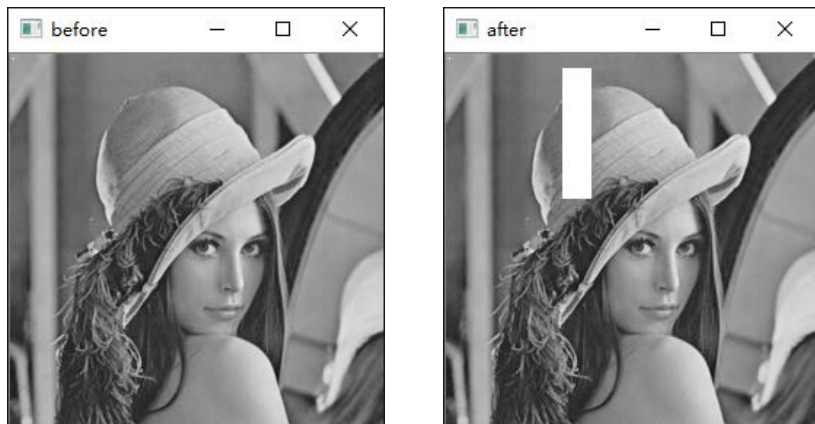


图 2-13 【例 2.9】程序的运行结果

同时，在控制台得到如下结果：

```
读取像素点 img.item(3,2)= 159
修改后像素点 img.item(3,2)= 255
```

2. 彩色图像

我们也可以使用函数 `item()` 和函数 `itemset()` 来访问和修改彩色图像的像素值，过程与操作灰度图像相似，不同之处在于需要补充通道信息。

函数 `item()` 访问 RGB 模式图像的像素值时，其语法格式为：

```
item(行,列,通道)
```

函数 `itemset()` 修改（设置）RGB 模式图像的像素值时，其语法格式为：

```
itemset(三元组索引值,新值)
```

需要注意，针对 RGB 图像的访问，必须同时指定行、列以及行列索引（通道），例如 `img.item(a,b,c)`。仅仅指定行和列是不可以的。

【例 2.10】 使用 Numpy 生成一个由随机数构成的三维数组，用来模拟一幅 RGB 色彩空间的彩色图像，并使用函数 `item()` 和 `itemset()` 来访问和修改它。

根据题目要求，编写代码如下：

```
import numpy as np
img=np.random.randint(10,99,size=[2,4,3],dtype=np.uint8)
print("img=\n",img)
print("读取像素点 img[1,2,0]=",img.item(1,2,0))
print("读取像素点 img[0,2,1]=",img.item(0,2,1))
print("读取像素点 img[1,0,2]=",img.item(1,0,2))
img.itemset((1,2,0),255)
img.itemset((0,2,1),255)
img.itemset((1,0,2),255)
print("修改后 img=\n",img)
```

```
print("修改后像素点 img[1,2,0]=",img.item(1,2,0))
print("修改后像素点 img[0,2,1]=",img.item(0,2,1))
print("修改后像素点 img[1,0,2]=",img.item(1,0,2))
```

运行上述程序，在控制台得到如下输出结果：

```
img=
[[[71 32 69]
  [43 73 67]
  [30 48 93]
  [79 52 69]]
 [[66 60 48]
  [68 17 10]
  [92 79 20]
  [70 14 85]]]
读取像素点 img[1,2,0]= 92
读取像素点 img[0,2,1]= 48
读取像素点 img[1,0,2]= 48
修改后 img=
[[[ 71  32  69]
  [ 43  73  67]
  [ 30 255  93]
  [ 79  52  69]]
 [[ 66  60 255]
  [ 68  17  10]
  [255  79  20]
  [ 70  14  85]]]
修改后像素点 img[1,2,0]= 255
修改后像素点 img[0,2,1]= 255
修改后像素点 img[1,0,2]= 255
```

在本例中，为了方便读者细致观察数组内的值，生成的数组尺寸较小。在实际使用中，读者可以利用随机函数生成更大尺寸的随机数组，并使用函数 `cv2.imshow()` 观察随机数组对应的彩色图像。

【例 2.11】生成一幅彩色图像，让其中的像素值均为随机数。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[256,256,3],dtype=np.uint8)
cv2.imshow("demo",img)
cv2.waitKey()
cv2.destroyAllWindows()
```

上述程序可以生成一幅 $256 \times 256 \times 3$ 的彩色图像，显示的图像如图 2-14 所示。由于本书为黑白印刷，所以为了更好地观察运行效果，请大家亲自上机运行程序。



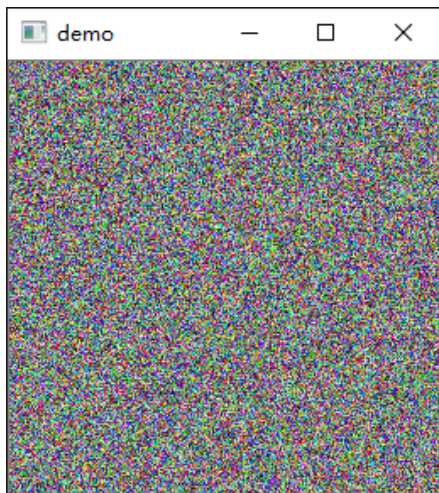


图 2-14 随机图像

【例 2.12】 读取一幅彩色图像，并对其像素进行访问、修改。

根据题目要求，编写代码如下：

```
import cv2
import numpy as np
img=cv2.imread("lenacolor.png")
cv2.imshow("before",img)
print("访问 img.item(0,0,0)=",img.item(0,0,0))
print("访问 img.item(0,0,1)=",img.item(0,0,1))
print("访问 img.item(0,0,2)=",img.item(0,0,2))
for i in range(0,50):
    for j in range(0,100):
        for k in range(0,3):
            img.itemset((i,j,k),255)    #白色
cv2.imshow("after",img)
print("修改后 img.item(0,0,0)=",img.item(0,0,0))
print("修改后 img.item(0,0,1)=",img.item(0,0,1))
print("修改后 img.item(0,0,2)=",img.item(0,0,2))
```

在本程序中，首先，使用 `item()` 函数读取了第 0 行第 0 列位置上的 B 通道、G 通道、R 通道三个通道上的像素值；接着，使用 `itemset()` 函数对该值进行了修改；最后，使用嵌套循环语句将位于“第 0 行到第 49 行”和“第 0 列到第 99 列”的交叉区域内的像素值设置为了 255，即让该区域显示为白色。

运行上述程序，得到如图 2-15 所示结果，其中左图是读取的原始图像，右图是修改后的图像。由于本书为黑白印刷，所以为了更好地观察运行效果，请大家亲自上机运行程序。

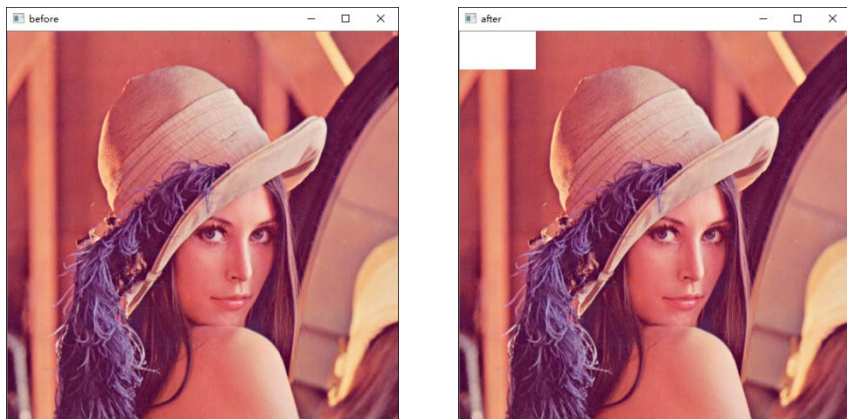


图 2-15 【例 2.12】程序的运行结果

同时，会在控制台显示如下结果：

```
访问 img.item(0,0,0)= 125
访问 img.item(0,0,1)= 137
访问 img.item(0,0,2)= 226
修改后 img.item(0,0,0)= 255
修改后 img.item(0,0,1)= 255
修改后 img.item(0,0,2)= 255
```

2.4 感兴趣区域 (ROI)

在图像处理过程中，我们可能会对图像的某一个特定区域感兴趣，该区域被称为感兴趣区域 (Region of Interest, ROI)。在设定感兴趣区域 ROI 后，就可以对该区域进行整体操作。例如，将一个感兴趣区域 A 赋值给变量 B 后，可以将该变量 B 赋值给另外一个区域 C，从而达到在区域 C 内复制区域 A 的目的。

例如，在图 2-16 中，假设当前图像的名称为 `img`，图中的数字分别表示行号和列号。那么，图像中的黑色 ROI 可以表示为 `img[200:400, 200:400]`。

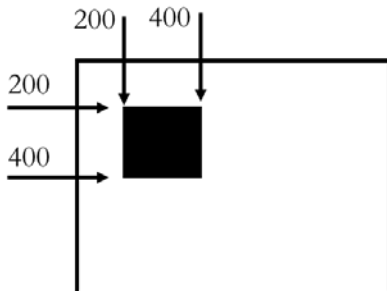


图 2-16 ROI 示例

通过以下语句，能够将图 2-16 中的黑色 ROI 复制到该区域右侧：

```
a=img[200:400,200:400]
img[200:400,600:800]=a
```

上述语句对图 2-16 的处理结果如图 2-17 所示，在该区域右侧得到了复制的 ROI。

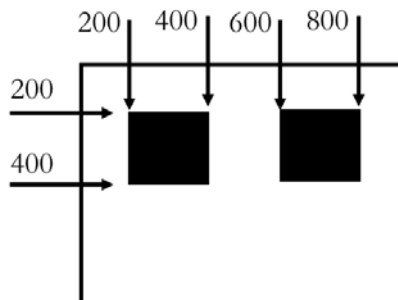


图 2-17 复制结果

【例 2.13】 获取图像 lena 的脸部信息，并将其显示出来。

根据题目要求，编写代码如下：

```
import cv2
a=cv2.imread("lena_color.png",cv2.IMREAD_UNCHANGED)
face=a[220:400,250:350]
cv2.imshow("original",a)
cv2.imshow("face",face)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，通过 `face=a[220:400,250:350]` 获取了一个 ROI，并使用函数 `cv2.imshow()` 将其显示了出来。

运行上述程序，会得到如图 2-18 所示的结果，其中左图是 lena 的原始图像，右图是从 lena 图像中获取的脸部图像。

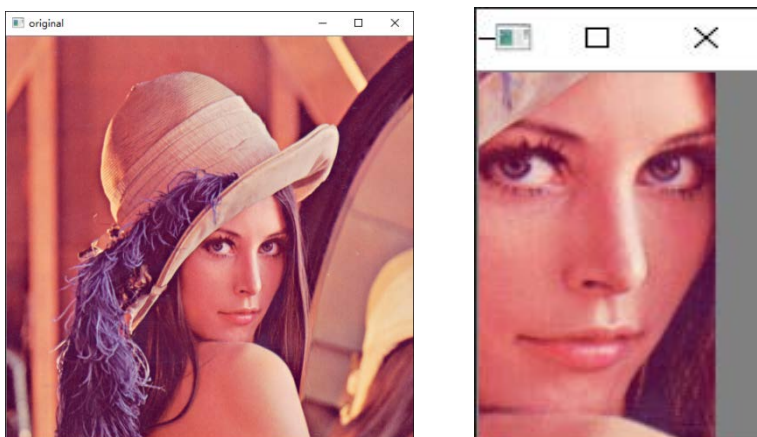


图 2-18 【例 2.13】程序的运行结果

【例 2.14】 对 lena 图像的脸部进行打码。

根据题目要求，编写代码如下：

```
import cv2
```

```
import numpy as np
a=cv2.imread("lenacolor.png",cv2.IMREAD_UNCHANGED)
cv2.imshow("original",a)
face=np.random.randint(0,256,(180,100,3))
a[220:400,250:350]=face
cv2.imshow("result",a)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，使用随机数生成的三维数组模拟了一幅随机图像，实现了对脸部图像的打码。

运行上述程序，得到如图 2-19 所示结果，其中左图是 lena 的原始图像，右图是脸部打码图像。

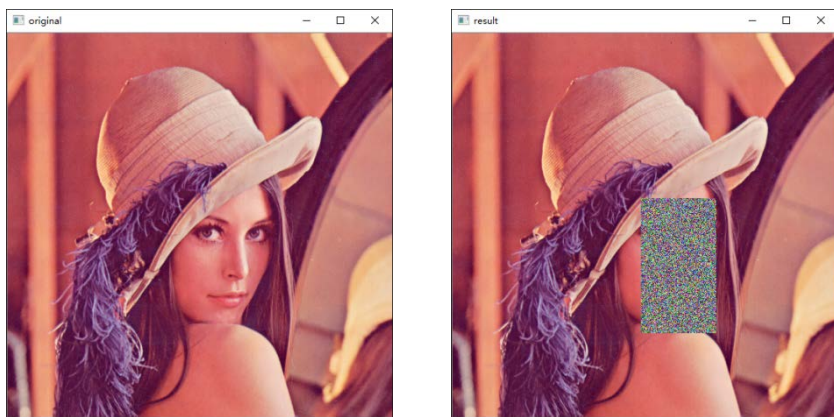


图 2-19 【例 2.14】程序的运行结果

【例 2.15】将一幅图像内的 ROI 复制到另一幅图像内。

根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread("lena512.bmp",cv2.IMREAD_UNCHANGED)
dollar=cv2.imread("dollar.bmp",cv2.IMREAD_UNCHANGED)
cv2.imshow("lena",lena)
cv2.imshow("dollar",dollar)
face=lena[220:400,250:350]
dollar[160:340,200:300]=face
cv2.imshow("result",dollar)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，得到如图 2-20 所示的结果。其中，左图是 lena 的原始图像，中间的图是 dollar 图像，右图是将 lena 图像内的 ROI 复制到 dollar 图像的效果。

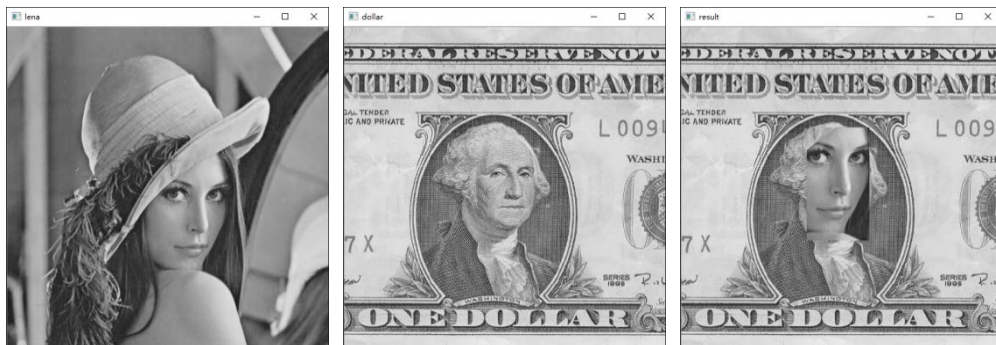


图 2-20 【例 2.15】程序的运行结果

2.5 通道操作

在 RGB 图像中，图像是由 R 通道、G 通道、B 通道三个通道构成的。需要注意的是，在 OpenCV 中，通道是按照 B 通道→G 通道→R 通道的顺序存储的。

在图像处理过程中，可以根据需要对通道进行拆分和合并。本节就来介绍如何对通道进行拆分和合并。

2.5.1 通道拆分

针对 RGB 图像，可以分别拆分出其 R 通道、G 通道、B 通道。在 OpenCV 中，既可以通过索引的方式拆分通道，也可以通过函数的方式拆分通道。

1. 通过索引拆分

通过索引的方式，可以直接将各个通道从图像内提取出来。例如，针对 OpenCV 内的 BGR 图像 img，如下语句分别从中提取了 B 通道、G 通道、R 通道。

```
b = img[ : , : , 0 ]
g = img[ : , : , 1 ]
r = img[ : , : , 2 ]
```

【例 2.16】编写程序，演示图像通道拆分及通道值改变对彩色图像的影响。

根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread("lenacolor.png")
cv2.imshow("lena1",lena)
b=lena[:, :, 0]
g=lena[:, :, 1]
r=lena[:, :, 2]
cv2.imshow("b",b)
cv2.imshow("g",g)
cv2.imshow("r",r)
lena[:, :, 0]=0
cv2.imshow("lenab0",lena)
```



```
lena[:, :, 1]=0
cv2.imshow("lenaBtoG", lena)
cv2.waitKey()
cv2.destroyAllWindows()
```

本例实现了通道拆分和通道值改变:

- 语句 `b=lena[:, :, 0]` 获取了图像 `img` 的 B 通道。
- 语句 `g=lena[:, :, 0]` 获取了图像 `img` 的 G 通道。
- 语句 `r=lena[:, :, 0]` 获取了图像 `img` 的 R 通道。
- 语句 `lena[:, :, 0]=0` 将图像 `img` 的 B 通道值设置为 0。
- 语句 `lena[:, :, 1]=0` 将图像 `img` 的 G 通道值设置为 0。

运行上述程序, 得到如图 2-21 所示的结果, 其中:

- 图(a)是原始图像 `lena`。
- 图(b)是原始图像 `lena` 的 B 通道图像 `b`。
- 图(c)是原始图像 `lena` 的 G 通道图像 `g`。
- 图(d)是原始图像 `lena` 的 R 通道图像 `r`。
- 图(e)是将图像 `lena` 中 B 通道值置为 0 后得到的图像。
- 图(f)是将图像 `lena` 中 B 通道值、G 通道值均置为 0 后得到的图像。

由于本书为黑白印刷, 所以为了更好地观察运行效果, 请大家亲自上机运行程序。

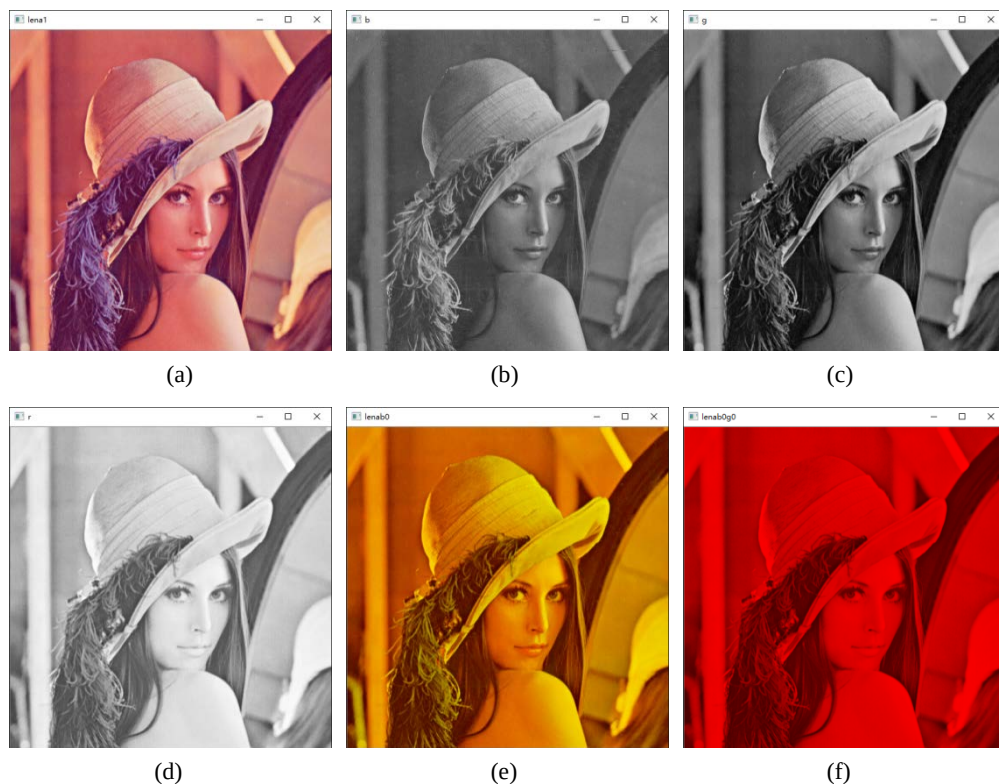


图 2-21 【例 2.16】程序的运行结果

2. 通过函数拆分

函数 `cv2.split()` 能够拆分图像的通道。例如，可以使用如下语句拆分彩色 BGR 图像 `img`，得到 B 通道图像 `b`、G 通道图像 `g` 和 R 通道图像 `r`。

```
b, g, r = cv2.split(img)
```

上述语句与如下语句是等价的：

```
b = cv2.split(a)[0]
g = cv2.split(a)[1]
r = cv2.split(a)[2]
```

【例 2.17】 编写程序，使用函数 `cv2.split()` 拆分图像通道。

根据题目要求，编写代码如下：

```
import cv2
lena = cv2.imread("lena_color.png")
b, g, r = cv2.split(lena)
cv2.imshow("B", b)
cv2.imshow("G", g)
cv2.imshow("R", r)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，得到如图 2-22 所示的三个通道图像，其中：

- 左图是 B 通道图像 `b`。
- 中间的图是 G 通道图像 `g`。
- 右图是 R 通道图像 `r`。

由于本书为黑白印刷，所以为了更好地观察运行效果，请大家亲自上机运行程序。

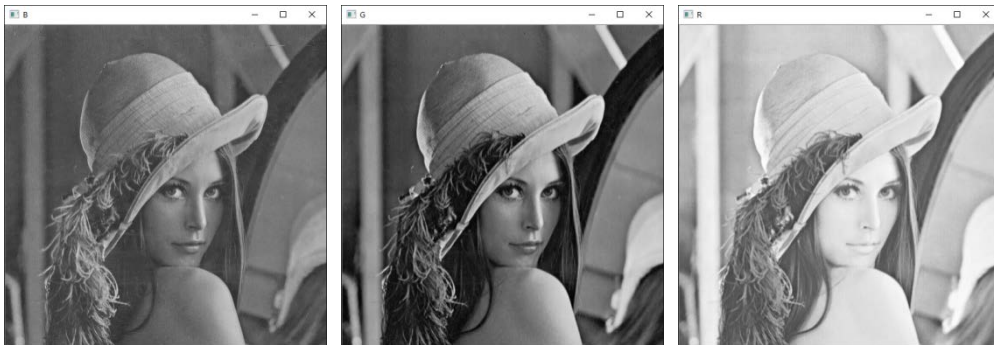


图 2-22 三个通道的图像

2.5.2 通道合并

通道合并是通道拆分的逆过程，通过合并通道可以将三个通道的灰度图像构成一幅彩色图像。函数 `cv2.merge()` 可以实现图像通道的合并，例如有 B 通道图像 `b`、G 通道图像 `g` 和 R 通道图像 `r`，使用函数 `cv2.merge()` 可以将这三个通道合并为一幅 BGR 的三通道彩色图像。其实现的语句为：

```
bgr=cv2.merge([b,g,r])
```

【例 2.18】 编写程序，演示使用函数 `cv2.merge()` 合并通道。

根据题目要求，编写代码如下：

```
import cv2
lena=cv2.imread("lenacolor.png")
b,g,r=cv2.split(lena)
bgr=cv2.merge([b,g,r])
rgb=cv2.merge([r,g,b])
cv2.imshow("lena",lena)
cv2.imshow("bgr",bgr)
cv2.imshow("rgb",rgb)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，首先对 BGR 图像进行了拆分，接下来又对其进行了两种不同形式的合并。

- 语句 `b,g,r=cv2.split(lena)` 对图像 `lena` 进行拆分，得到 `b`、`g`、`r` 这三个通道。
- 语句 `bgr=cv2.merge([b,g,r])` 对通道 `b`、`g`、`r` 进行合并，合并顺序为 B 通道→G 通道→R 通道，得到图像 `bgr`。
- 语句 `rgb=cv2.merge([r,g,b])` 对通道 `r`、`g`、`b` 进行合并，合并顺序为 R 通道→G 通道→B 通道，得到图像 `rgb`。

运行上述程序，得到如图 2-23 所示的图像，其中：

- 左图是原始图像 `lena`。
- 中间的图是 `lena` 图像经过通道拆分、合并后得到的 BGR 通道顺序的彩色图像 `bgr`。
- 右图是 `lena` 图像经过通道拆分、合并后得到的 RGB 通道顺序的彩色图像 `rgb`。

由于本书为黑白印刷，所以为了更好地观察运行效果，请大家亲自上机运行程序。

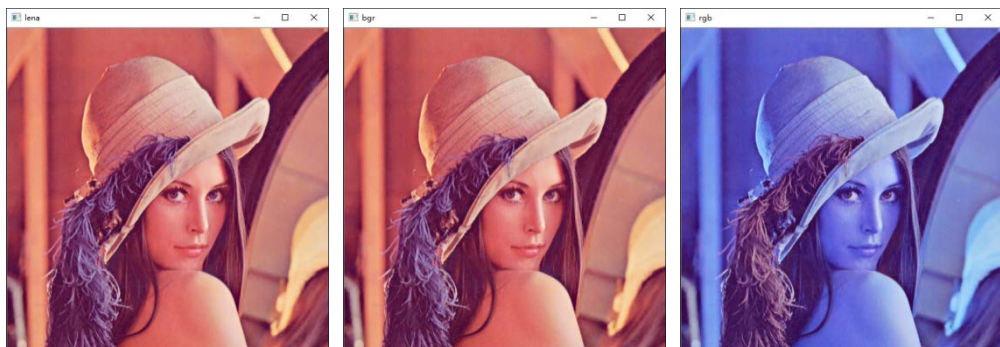


图 2-23 【例 2.18】程序的运行结果

通过本例可以看出，改变通道顺序后，图像显示效果会发生变化。

2.6 获取图像属性

在图像处理过程中，经常需要获取图像的属性，例如图像的大小、类型等。这里介绍几个常用的属性。

- **shape**: 如果是彩色图像，则返回包含行数、列数、通道数的数组；如果是二值图像或者灰度图像，则仅返回行数和列数。通过该属性的返回值是否包含通道数，可以判断一幅图像是灰度图像（或二值图像）还是彩色图像。
- **size**: 返回图像的像素数目。其值为“行×列×通道数”，灰度图像或者二值图像的通道数为 1。
- **dtype**: 返回图像的数据类型。

【例 2.19】编写程序，观察图像的常用属性值。

根据题目要求，编写代码如下：

```
import cv2
gray=cv2.imread("lena.bmp",0)
color=cv2.imread("lenacolor.png")
print("图像 gray 属性: ")
print("gray.shape=",gray.shape)
print("gray.size=",gray.size)
print("gray.dtype=",gray.dtype)
print("图像 color 属性: ")
print("color.shape=",color.shape)
print("color.size=",color.size)
print("color.dtype=",color.dtype)
```

在本例中，分别读取了灰度图像 `gray` 和彩色图像 `color`，并分别观察了它们的 `shape`、`size`、`dtype` 属性。

运行程序，控制台会输出如下结果：

```
图像 gray 属性:
gray.shape= (256, 256)
gray.size= 65536
gray.dtype= uint8
图像 color 属性:
color.shape= (512, 512, 3)
color.size= 786432
color.dtype= uint8
```